spread across various experts within the organisation. After all, standardising, managing and getting software out of the door is a complex exercise.

Modern enterprises expect near-real time (if not real-time) access to applications as they are developed. Gone are the days waiting for a URL update for six months because it's the norm. Integrating security within CI and CD processes has resulted in the evolution of DevSecOps. Figure 1 shows the integration of typical software development, testing and deployment, and the operational journey of code. If planned along with the application design and architecture, DevSecOps can be automated more quickly as all value propositions can focus only on gaining efficiency and forego all repetitive mundane tasks of building, testing and deploying.

The security (Sec) part of DevSecOps automates all types of testing as part of code build and not just security. The code being validated against testing first goes through unit, functional and integration testing to ensure it does what is expected of it, and then the security is validated before the code is deployed.

Now that we have established what DevSecOps is and the automation of code development, testing, deployment and monitoring aspects, let's look at some of the key benefits of adopting AI in DevSecOps:

- Improved security and reporting
- Improved reliability and resiliency of automation
- Ease of access to contextual data across the development life cycle
- Enhanced resource management with ability to respond automatically
- Ambiguity and anomaly detection with reliable pattern analysis
- Improved collaboration with AI connecting the knowledge bases

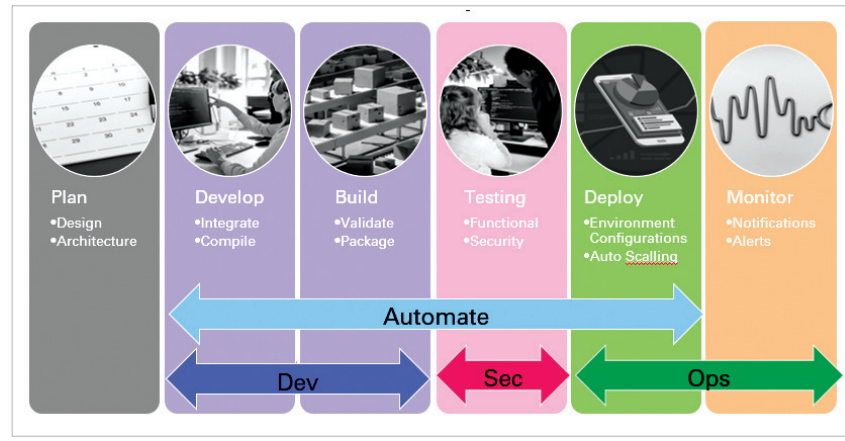

Figure 1: Automated DevSecOps orchestration

Testing type	Description	Responsibility of
Unit testing	Test individual modules of application in isolation to validate code is executing as intended	Developer
Functional testing	Test a specific functionality in the system to validate the code is executing and responding to user actions	Tester
System acceptance testing (SAT)	Also known as integration testing that involves overall testing of systems of many sub-systems/ applications or elements.	Domain experts and business users
User acceptance testing (UAT)	Validate application life cycle before go-live	Business users
Security testing	Process intended to reveal flaws in the security mechanisms	Security tester

Table 1: Types of testing and who is responsible for them in an organisation

The power of AI-driven chatbots

Chatbots have become a common tool for everyday users of technology across all types of devices. The younger generation thrives in the world of social media apps that are designed to respond to the briefest of interests with specific information. By integrating knowledge repositories, information security policies and regulations, test databases and generative AI capabilities, a powerful chatbot can offer valuable insights. For example, a developer can request the bot to compile specific compliance mandates so the person can understand how to code according to them. Or a tester can request what types of tests have been performed historically against a specific code module and what the outcome of those test executions are, to help validate the stability of the code. This can also be used by project managers to learn the effort and time it has taken for specific modules to be updated in the past, and forecast more accurately the release date of the application.

When only a select few individuals in an organisation know how to respond to specific scenarios or incidents, it is typically known as tribal knowledge. This knowledge can be automated by creating a feedback loop across the DevSecOps pipeline, which is data-driven. Here, events are funnelled to a previous process with AI to help learn or adapt to new information.